

Security Overview (Plaid)

Scope

- Application: Personal Finances (personal-use app; single developer/administrator; no employees/contractors)
- Components:
 - Web: React + Vite
 - API: NestJS (Node.js)
 - Database: PostgreSQL via Prisma ORM
 - Data provider: Plaid (Link + Transactions Sync)

Data flow summary

1. User authenticates to the API with email/password.
2. API issues a short payload JWT (Bearer token) for API calls.
3. User links a financial institution via Plaid Link.
4. API exchanges Plaid `public_token` for an `access_token` server-side.
5. API uses the `access_token` to sync transactions and store normalized results in PostgreSQL.

Data stored

- User: email, `passwordHash` (bcrypt), timestamps.
- Plaid linkage: Plaid item identifier and Plaid access token (treated as a secret; never returned to clients).
- Finance data: account identifiers/metadata and transaction records required to display dashboards.

Key security controls

- Authentication:
 - Passwords are never stored in plaintext; bcrypt hashing is used.
 - API access uses JWT Bearer tokens.
- Authorization:
 - Finance, Plaid, and sync endpoints require authentication.
 - Data queries are scoped by `userId`.
- Transport security:
 - Production access is intended to be served over HTTPS (handled by the hosting provider / reverse proxy).
 - API calls to Plaid occur server-to-server.
- Secrets management:
 - Secrets are provided via environment variables / hosting secret manager.
 - Secrets are not committed to source control (local secret env files are ignored; example/template env files may be committed).

- Input validation:
 - API uses request DTO validation and global validation (whitelisting, type transforms).
- Logging:
 - Access tokens and sensitive credentials are not logged.

Operational notes (solo developer)

- Administrative access is limited to the single developer account.
- MFA is enabled for critical accounts (email, source control, hosting provider, Plaid dashboard).